Software Requirements Specification (SRS)

Animal Rescue Management System

Course: CSE336 — Software Design Patterns
Project: Animal Rescue Management System
Version: 1.0
Document Type: Software Requirements Specification

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements of the Animal Rescue Management System.

The document describes the system's objectives, main features, intended users, architecture, and design components. It provides a common understanding between the development team and the intended users of the system.

It also serves as a reference for implementation, testing, and future development.

1.2 Project Definition

The Animal Rescue Management System is a Java-based desktop application designed to help animal rescue organizations manage their daily activities.

The system provides a centralized application for managing rescued animals, potential adopters, donations, volunteers, rescue events, and communication.

Instead of managing these activities through separate records, the organization can use the system's graphical interface to organize information and perform common management operations.

1.3 Project Purpose

The main purpose of the project is to simplify and organize the management of animal rescue activities.

The system aims to:

- Maintain organized records of rescued animals.
- Manage information about potential adopters.
- Record and organize donations.
- Maintain volunteer information and availability.
- Organize rescue events and volunteer registrations.
- Support communication through different notification channels.
- Provide a centralized interface for accessing management modules.
- Apply software design patterns to improve code organization, flexibility, and maintainability.

1.4 Scope

The system covers the main administrative activities of an animal rescue organization.

The project includes the following modules:

1. Animal Management
2. Adopter Management
3. Donation Management
4. Volunteer Management
5. Rescue Event Management
6. Communication Management
7. User Management
8. Main Control Panel

The application is developed as a Java desktop application using Swing for its graphical user interface.

The current implementation uses in-memory data storage. Permanent database storage is not included in the documented current scope.

1.5 Definitions and Abbreviations

Term	Definition
SRS	Software Requirements Specification
CRUD	Create, Read, Update, Delete
MVC	Model-View-Controller
UML	Unified Modeling Language
Animal	An animal registered and managed by the rescue organization
Adopter	A person interested in adopting an animal
Donor	A person or entity providing a donation
Volunteer	A person who offers assistance to the organization
Rescue Event	An activity organized by the rescue organization
Repository	A component responsible for managing access to stored application data
Observer	An object that receives notifications when a relevant event occurs
Strategy	A design pattern that allows interchangeable algorithms or behaviors

1.6 Intended Audience

This document is intended for:

- The development team responsible for implementing the system.
- The project instructor and evaluators.
- Rescue organization staff who may use the application.
- Testers responsible for checking system functionality.
- Future developers who may extend the application.

2. Overall Description

2.1 Product Perspective

The Animal Rescue Management System is a standalone Java desktop application.

It is organized into separate modules, each responsible for a specific part of the organization's activities.

The modules are accessed through a main control panel. The application follows an MVC-style architecture to separate the user interface, application logic, and domain data.

2.2 Product Functions

The system provides the following main functions:

- Registering and managing rescued animal records.
- Managing adopter information.
- Recording different types of donations.
- Registering and managing volunteers.
- Organizing rescue events.
- Registering volunteers for events.
- Matching volunteers according to skills or availability.
- Supporting communication through different channels.
- Managing system user records.
- Navigating between the different modules through the control panel.

2.3 User Classes

1. Rescue Staff Member

Responsible for maintaining animal records and updating animal information.

2. Adoption Coordinator

Responsible for maintaining information about potential adopters and managing adoption-related records.

3. Donation Manager

Responsible for recording donations and maintaining donation information.

4. Volunteer Coordinator

Responsible for registering volunteers, maintaining their skills and availability, and selecting suitable volunteers.

5. Event Coordinator

Responsible for organizing rescue events and managing volunteer registrations.

6. System Administrator

Responsible for managing system user records and accessing administrative functionality.

These are intended user roles for describing system requirements; they do not necessarily represent separate authenticated permission levels in the current implementation.

2.4 Operating Environment

The system is designed to operate in the following environment:

- Operating System: Windows, Linux, or macOS with Java support.
- Programming Language: Java.
- Java Version: JDK 17 or newer.
- User Interface: Java Swing.
- Build Tool: Apache Maven.
- Development Environment: IntelliJ IDEA, Eclipse, or another Java-compatible IDE.

2.5 Design and Implementation Constraints

- The application is developed using Java.
- The graphical interface is implemented using Swing.
- The application follows an MVC-style organization.
- The system uses repositories to organize data operations.
- The current DataStore implementation stores data in memory.
- The application applies software design patterns as part of the course requirements.

- The current system does not provide permanent data persistence after the application closes.

2.6 Assumptions and Dependencies

- Users have access to a computer capable of running a Java application.
- Java is installed in the execution environment.
- The application is launched through its main entry point.
- Users interact with the application through its graphical interface.
- The system's data remains available during the running session, subject to the limitations of in-memory storage.

3. Functional Requirements

Functional requirements describe the operations and services that the system is expected to provide.

3.1 Animal Management

FR-01 — Add Animal

The system shall allow authorized staff to add a new animal record containing relevant information such as name, species, age, health status, adoption availability, and description.

FR-02 — View Animals

The system shall allow staff to view registered animal records.

FR-03 — Update Animal

The system shall allow staff to modify existing animal information.

FR-04 — Delete Animal

The system shall allow staff to delete an animal record.

FR-05 — Update Animal Status

The system shall allow staff to update an animal's health status and adoption availability.

3.2 Adopter Management

FR-06 — Add Adopter

The system shall allow staff to register a new adopter and store relevant adopter information.

FR-07 — View Adopters

The system shall allow staff to view registered adopter records.

FR-08 — Update Adopter

The system shall allow staff to update existing adopter information.

FR-09 — Delete Adopter

The system shall allow staff to delete adopter records.

3.3 Donation Management

FR-10 — Record Donation

The system shall allow donation managers to record donation information.

FR-11 — Support Donation Types

The system shall support different donation types, including online, cheque, and in-kind donations.

FR-12 — Add Donation Decorations

The system shall support optional donation features such as recurring donations and "in memory of" dedications.

FR-13 — View Donation Records

The system shall allow users to access recorded donation information.

3.4 Volunteer Management

FR-14 — Register Volunteer

The system shall allow volunteer coordinators to register volunteers and store their relevant information.

FR-15 — Maintain Volunteer Information

The system shall allow volunteer information, including skills and availability, to be maintained.

FR-16 — Match Volunteers

The system shall support selecting volunteers based on matching strategies such as skill matching and availability matching.

3.5 Rescue Event Management

FR-17 — Create Event

The system shall allow users to create rescue events and store their information.

FR-18 — Update Event

The system shall allow users to update existing event information.

FR-19 — Register Volunteer for Event

The system shall allow volunteers to be registered for specific rescue events.

FR-20 — View Event Information

The system shall allow users to access event information and registration records.

3.6 Communication Management

FR-21 — Select Communication Channel

The system shall support selecting different communication strategies, including email, SMS, and social media posts.

FR-22 — Manage Recipients

The system shall support different recipient types for communication operations.

FR-23 — Respond to Relevant Events

The system shall provide observer components intended to respond to relevant animal-status changes, donation activity, and event updates.

Note: Complete integration of these observer workflows must be verified in the application before claiming that all notifications are triggered automatically.

3.7 User Management

FR-24 — Manage Users

The system shall provide functionality for managing system user records.

The exact authentication and permission-management behavior should be specified according to the final implemented user-management workflow.

3.8 Control Panel

FR-25 — Access Management Modules

The system shall provide a main control panel that allows users to navigate to the available management modules.

4. Non-Functional Requirements

Non-functional requirements describe the quality attributes and operational expectations of the system.

4.1 Usability

- The graphical interface should be understandable and easy to navigate.
- Management modules should be accessible through the main control panel.
- Data-entry screens should clearly identify the information required from users.

4.2 Maintainability

- The system should maintain separation between models, views, and controllers.
- Classes should have clear and focused responsibilities.
- Repositories and services should organize data operations and application behavior.

- Design patterns should be used to support extensibility and reduce unnecessary coupling.

4.3 Performance

- Common operations such as viewing, adding, updating, and deleting records should complete within a reasonable time for the expected project data size.
- Navigation between management modules should be responsive.

4.4 Reliability

- The application should handle invalid user input without unexpectedly terminating.
- Data operations should maintain consistent records during the application session.

4.5 Portability

- The application should be executable on operating systems that support the required Java version.

4.6 Data Management

- The current application uses in-memory storage.
- Data is available during the active application session.
- Data persistence across application restarts is outside the current implementation.

5. System Design and Main Components

5.1 Architectural Design

The system follows the Model-View-Controller (MVC) architectural pattern.

MVC separates the application into three main parts:

Model

The Model represents the application's data and business entities.

Examples include:

- Animal
- Adopter
- Donor
- Volunteer
- RescueEvent
- EventRegistration
- Donation
- User

View

The View represents the graphical interface presented to users.

It contains Swing screens for the different management modules.

Controller

The Controller handles user interactions and coordinates communication between the View, Model, repositories, and services.

This separation makes the application easier to organize, maintain, and extend.

5.2 Main Software Components

1. Model Component

Package: `com.rescue.model`

Responsible for defining the main application entities, their attributes, and their relationships.

2. Repository Component

Package: `com.rescue.repo`

Responsible for organizing data-access operations such as creating, finding, updating, and deleting records.

3. Controller Component

Package: `com.rescue.controller`

Responsible for processing user actions and coordinating application operations.

4. View Component

Package: `com.rescue.view`

Responsible for displaying the application's graphical user interface using Java Swing.

5. Service Component

Package: `com.rescue.service`

Responsible for application-level operations and coordination, including communication and volunteer-matching behavior.

6. Factory Component

Package: `com.rescue.factory`

Responsible for creating objects using factory-based object creation.

The current implementation includes `DonationFactory` .

7. Observer Component

Package: `com.rescue.service.observer`

Contains observer-related interfaces and classes intended to support event-based notifications.

8. Strategy Component

Package: `com.rescue.service.strategy`

Contains interchangeable strategies for communication channels and volunteer matching.

6. Design Patterns

The project applies six design patterns in Phase 1.

6.1 MVC — Model View Controller

MVC separates application data, graphical interfaces, and user interaction handling.

In this project:

- Models represent system entities.
- Views display the management screens.
- Controllers handle user actions.

6.2 Strategy Pattern

The Strategy pattern allows different algorithms or behaviors to be selected without changing the code that uses them.

In this project, it is used for:

- Communication channel strategies.
- Volunteer matching strategies.

Examples include `EmailStrategy`, `SmsStrategy`, `SocialPostStrategy`, `SkillMatchingStrategy`, and `AvailabilityMatchingStrategy`.

6.3 Observer Pattern

The Observer pattern allows interested objects to receive notifications when relevant events occur.

The project contains observer interfaces and implementations related to animal-status changes, donations, and event updates.

6.4 Decorator Pattern

The Decorator pattern allows additional behavior or features to be added to an object without modifying its original implementation.

In this project, donation decorators provide optional features such as recurring donations and "in memory of" dedications.

6.5 Factory Pattern

The Factory pattern centralizes object creation.

In this project, `DonationFactory` creates different donation implementations, such as online, cheque, and in-kind donations.

6.6 Singleton Pattern

The Singleton pattern ensures that a class provides a shared instance.

In this project, `DataStore` and `AppConfig` use the Singleton pattern to provide shared application-wide objects.

7. Data Requirements

The system manages information related to the following entities:

Entity	Description
Animal	Stores rescued animal information and status.
Adopter	Stores information about potential adopters.
Donor	Represents a donor providing contributions.
Donation	Represents a contribution and its type or optional features.
Volunteer	Stores volunteer information, skills, and availability.
RescueEvent	Stores information about rescue events.
EventRegistration	Represents a volunteer's registration for an event.
User	Represents a system user.
SentMessage	Represents a recorded communication message.

The current implementation uses in-memory data storage through the application's data store and repositories.

8. External Interface Requirements

8.1 User Interface

The system provides a graphical user interface built with Java Swing.

The interface is organized into management screens that users can access through the main control panel.

8.2 Hardware Interface

The system requires a computer capable of running a Java desktop application.

No specialized hardware is required by the current project scope.

8.3 Software Interface

The system depends on:

- Java Runtime Environment / JDK.
- Java Swing libraries.
- Maven for project building and dependency management.

The current project does not specify an external database integration.

9. Future Enhancements

The following features may be considered in future versions, but are not claimed as implemented in the current system:

- Permanent database storage.
- Advanced search and filtering.
- More detailed adoption workflows.
- Improved reporting and statistics.
- Expanded notification integrations.
- More advanced user authentication and permissions.

Additional design patterns may also be introduced during Phase 2 as required by the course.

10. Conclusion

The Animal Rescue Management System provides a structured desktop application for organizing the main activities of an animal rescue organization.

Through its separate management modules, MVC-style architecture, repository organization, and six Phase 1 design patterns, the project aims to provide maintainable and extensible software.

This SRS document defines the system's scope, intended users, functional requirements, non-functional requirements, architecture, and main software components. It serves as a reference for implementation, evaluation, testing, and future development.